StatLean

A Lean 4 Formalization of Statistics

Gan Tong

Department of Statistics and Data Science
National University of Singapore

March 2026

Why Formalize Statistics?

The Problem

- In the LLM era, training on statistical proofs requires high-quality formal data
- Lean 4 + Mathlib provide a mature platform for mathematics
- But **no comprehensive statistics library** exists yet

Our Approach

- **AI-assisted formalization**: LLM-driven proof search with human guidance
- Build on **Mathlib** (measure theory, probability, analysis)
- **Automated curation**: verified lemmas auto-indexed into reusable library

Benefits

1. **Reusability**: proved lemmas compose into larger results
2. **Discovery**: formalization reveals hidden structure and missing conditions

Project Overview

Tech Stack

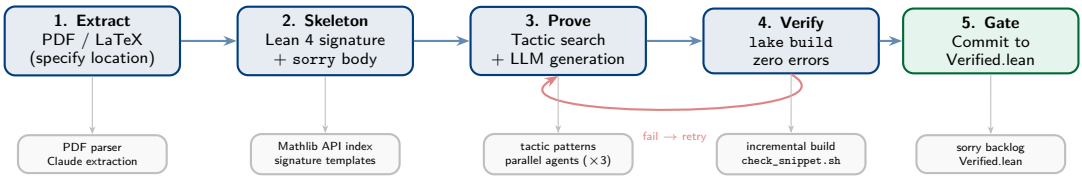
- Lean 4.28 + Mathlib (latest)
- Lake build system

Scale

45	Lean source files
14,400+	lines of Lean code
490+	formal declarations
38	fully verified modules (zero `sorry`)
6	remaining `sorry` gaps

`lake build Statlean.Verified` → zero warnings

End-to-End Formalization Pipeline



Input: PDF/LaTeX with specified theorem location,
or direct text description of the theorem to formalize
Output: machine-verified Lean 4 proof

Automation: Steps 1–4 driven by Claude Code
Human: review gate at Step 5, mathematical guidance

Step 1–2: From Textbook to Lean Skeleton

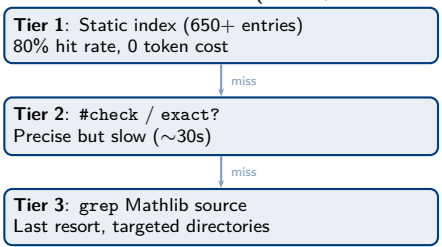
Theorem Extraction

1. Parse PDF / LaTeX source
2. Identify theorem statement, hypotheses, notation
3. Map to Mathlib types and conventions
4. Resolve ambiguity with textbook context

Signature Design Principles

- Follow Mathlib conventions (universe-polymorphic, typeclass args)
- All conditions explicit (no "clearly integrable")

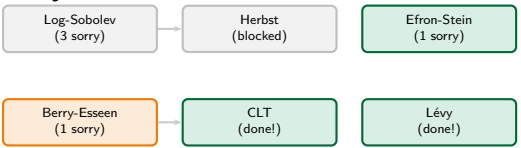
Mathlib API Search (3-tier, mandatory)



Avoids redundant Mathlib searches — the biggest token cost in LLM-based proving.

Step 3–4: Prove and Verify

Sorry DAG Scheduler

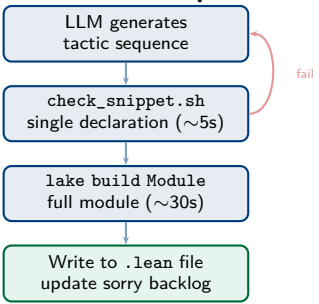


- **Leaves first**: attack goals with no downstream dependencies
- **Skip blocked**: don't waste cycles on dependency chains
- **Parallel**: up to 3 agents on independent goals

Tactic Pattern Library

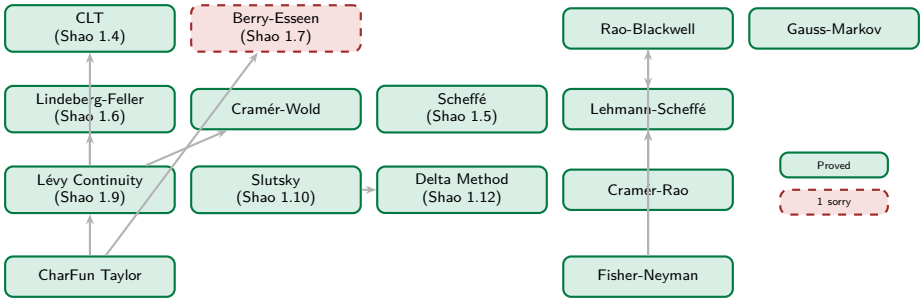
- Indexed by goal shape (e.g. "$\int f\, d\mu = 0$" → `integral_condExp`)
- Only verified patterns (`lake build` passed)

Verification Loop



Key: sub-lemmas written to disk *immediately* after verification — context exhaustion won't lose progress.

Theorem Dependency Map



AI-Assisted Proving: Results

What Claude Code Does

Task	Method
Find Mathlib API	3-tier index search
Generate proof	Pattern library + LLM
Verify	`lake build` (incremental)
Manage sorry DAG	YAML backlog + priority
Parallelize	Up to 3 concurrent agents
Persist progress	Immediate disk write

What the Human Does

- Mathematical insight for hard blockers
- Decompose complex goals into sub-lemmas
- Review and approve commits
- Decide attack priority

Proof Difficulty Grading and Cost

Sorry Grading System (calibrated from StatLean attack logs)

	Type	Lines	Time	Tokens	Opus $
S	Thin wrapper	1–5	2–5 min	5–20K	0.2–0.7
A	Compose APIs	10–30	5–15 min	20–50K	0.7–1.7
B	Arithmetic	30–80	15–40 min	50–100K	1.7–3.3
C	Structural	80–200	0.5–1.5 hr	100–160K	3.3–5.3
D	Infra missing	150–400	1–3 hr	150–300K	5–10
E	Research-level	200–700+	3–10+ hr	300–500K+	10–17+

Opus 4.6 blended: \$33/M tok (70% in \$15 + 30% out \$75)

Grading Rule: grep goal in API index → exact match **S**, composable **A**, algebra **B**, new lemmas **C**, API missing **D**, route unclear **E**

Validated Examples

Theorem		Tok	
Kolmogorov 0-1	S	95K	✓
Multivar. CLT	A	141K	✓
Lyapunov	B	157K	✓
Kolmogorov max	C	164K	1 sorry
Glivenko-Cantelli	C	156K	1 sorry
CLT	D	200K	✓
Lindeberg-Feller	D	300K	✓
Lévy cont.	D	250K	✓
Hypercontract.	E	400K	×
Stieltjes inv.	E	300K	×

S–D: \$0.2–\$10 per theorem

E: \$10+ with no success guarantee

Future I: LLM Benchmark for Theorem Proving

Goal: evaluate LLM proving on *real* statistics theorems

18 Problems (from StatLean verified theorems)

Grade	#	Type	Example
S–A	5	Wrapper / compose	Slutsky, Gauss-Markov
B	4	Arithmetic	Rao-Blackwell, MSE
C	4	Structural	Scheffé, NP lemma
D	4	Infra missing	Lindeberg-Feller, USLLN
E	1	Research-level	Berry-Esseen (unsolved)

13 Models across 7 providers

Anthropic (Opus/Sonnet/Haiku) · OpenAI (GPT-5.2/o3) · Google (Gemini 3.1 Pro/2.5 Flash) · DeepSeek (V3.2 Chat/Reasoner) · Qwen 3.5 · MiniMax M2.5 · Llama 4 · **DeepSeek-Prover-V2**

Experiment Design

Condition	Content
Bare	theorem + imports only
Skill	+ tactic patterns + API index
Budget	1-round vs. 4-round retry

Matrix: $13 \times 18 \times 2 \times 2 \times N = 936N$ runs

Metrics: success rate, cost/theorem, Pass@k, first-pass rate, failure taxonomy

Future II: Web Platform for Community Contributions

Vision: lower the barrier — anyone can contribute verified proofs via web interface

Workflow A: Bring Your Own Theorem

1. Upload PDF/LaTeX with target theorem (specify name and location), *or* directly input theorem statement
2. Enter API key (BYOK — zero hosting cost)
3. Receive **estimated time & token cost**; confirm
4. Monitor formalization logs in real time
5. Download completed `.lean` file + reusable lemmas contributed to StatLean

Workflow B: Attack Open Sorry Goals

1. Browse open `sorry` goals (prioritized by S–E)
2. Select a goal, enter API key
3. Receive estimated cost; confirm
4. Monitor formalization logs in real time
5. Download result; reusable parts auto-contributed

Key Design: **BYOK** (user's API key) · **Sandboxed** (isolated Lean env) · **Reproducible** (full logs) · **Open data** (traces for research)

GitHub: `github.com/mockingbird-gan/statlean4`